

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”
ІНСТИТУТ КОМП’ЮТЕРНИХ ТЕХНОЛОГІЙ, АВТОМАТИКИ ТА
МЕТРОЛОГІЇ
КАФЕДРА КОМП’ЮТЕРИЗОВАНИХ СИСТЕМ АВТОМАТИКИ



ЗВІТ

про виконання лабораторної роботи № 5
з навчальної дисципліни: «Організація баз даних та знань»
Варіант № 102

Виконав:

студент групи ІР-24
Шийка Андрій

Прийняла:

Лагун І. І.

ЛЬВІВ – 2025

Завдання

1. **GitHub:** https://github.com/skrix/databases_p1_labs_iot_nulp/pull/5
2. Додати до БД 1 додаткову довільну таблицю і зв'язати з іншою існуючою таблицею зв'язком 1:М. Однак для забезпечення цілісності значень використати **тригери** замість фізичного зовнішнього ключа.

[source](#)

```
19 def upgrade():
20     # 1. Create table without foreign keys
21     op.execute("""
22         CREATE TABLE user_notes (
23             id INT AUTO_INCREMENT PRIMARY KEY,
24             user_id INT NOT NULL,
25             note TEXT NOT NULL,
26             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
27         );
28     """)
29
30     # 2. Trigger: Validate user exists on INSERT
31     op.execute("""
32         CREATE TRIGGER trg_user_notes_before_insert
33         BEFORE INSERT ON user_notes
34         FOR EACH ROW
35         BEGIN
36             IF (SELECT COUNT(*) FROM users WHERE id = NEW.user_id) = 0 THEN
37                 SIGNAL SQLSTATE '45000'
38                 SET MESSAGE_TEXT = 'Invalid user_id: user does not exist.';
39             END IF;
40         END;
41     """)
42
43     # 3. Trigger: Validate user exists on UPDATE
44     op.execute("""
45         CREATE TRIGGER trg_user_notes_before_update
46         BEFORE UPDATE ON user_notes
47         FOR EACH ROW
48         BEGIN
49             IF NEW.user_id IS NULL THEN
50                 SIGNAL SQLSTATE '45000'
51                 SET MESSAGE_TEXT = 'user_id cannot be NULL.';
52             END IF;
53
54             IF (SELECT COUNT(*) FROM users WHERE id = NEW.user_id) = 0 THEN
55                 SIGNAL SQLSTATE '45000'
56                 SET MESSAGE_TEXT = 'Invalid user_id on UPDATE: user does not exist.';
57             END IF;
58         END;
59     """)
60
61     # 4. Trigger: Prevent deletion of user with notes
62     op.execute("""
63         CREATE TRIGGER trg_users_before_delete
64         BEFORE DELETE ON users
65         FOR EACH ROW
66         BEGIN
67             IF (SELECT COUNT(*) FROM user_notes WHERE user_id = OLD.id) > 0 THEN
68                 SIGNAL SQLSTATE '45000'
69                 SET MESSAGE_TEXT = 'Cannot delete user: user_notes exist.';
70             END IF;
71         END;
72     """)
```

3. Збережені процедури:

- a. Забезпечити параметризовану вставку нових значень у довільну таблицю.

Users:

The screenshot shows a REST client interface with a POST request to `{{base_url}}/api/stored-procedures/generic-insert`. The request body is a JSON object:

```
1 {
2   "table": "users",
3   "columns": "first_name, last_name, email, password, driver_license, gender, dob",
4   "values": "'Jane', 'Doe', 'jane.doe@test.com', 'password123', 'XYZ-789-012', 'female', '1992-03-15'"
5 }
```

The response is a JSON object:

```
1 {
2   "message": "Successfully inserted 1 row(s) into users",
3   "rows_affected": 1,
4   "status": "success",
5   "table_name": "users"
6 }
```

The status bar indicates **201 CREATED** with a response time of 86 ms and a body size of 302 B.

Vehicles:

The screenshot shows a REST client interface with a POST request to `{{base_url}}/api/stored-procedures/generic-insert`. The request body is a JSON object:

```
1 {
2   "table": "vehicles",
3   "columns": "make, model, year, vin, body, plate",
4   "values": "'Mercedes', 'C-Class', 2024, 'WDD12345678901234', 'sedan', 'MERC-C24'"
5 }
```

The response is a JSON object:

```
1 {
2   "message": "Successfully inserted 1 row(s) into vehicles",
3   "rows_affected": 1,
4   "status": "success",
5   "table_name": "vehicles"
6 }
```

The status bar indicates **201 CREATED** with a response time of 20 ms and a body size of 308 B.

Rentings:

The screenshot shows a REST client interface with a POST request to `{{base_url}}/api/stored-procedures/generic-insert`. The request body is a JSON object:

```
1 {
2   "table": "rentings",
3   "columns": "user_id, vehicle_id, start_at, end_at",
4   "values": "2, 2, '2025-12-10 14:00:00', '2025-12-15 14:00:00'"
5 }
```

The response is a JSON object:

```
1 {
2   "message": "Successfully inserted 1 row(s) into rentings",
3   "rows_affected": 1,
4   "status": "success",
5   "table_name": "rentings"
6 }
```

The status bar indicates **201 CREATED** with a response time of 8 ms and a body size of 308 B.

- b. Забезпечити реалізацію зв'язку М:М між 2ма таблицями, тобто вставити в стикувальну таблицю відповідну стрічку за реально-існуючими значеннями (напр. surname, name) в цих основних таблицях.

RentingsFines:

The screenshot shows a REST client interface with a POST request to `{{base_url}} /api/stored-procedures/join_insert`. The request body is a JSON object with the following structure:

```
1 {
2   "left_table": "rentings",
3   "left_lookup_col": "id",
4   "left_lookup_val": "1",
5   "right_table": "fines",
6   "right_lookup_col": "id",
7   "right_lookup_val": "2",
8   "join_table": "rentings_fines",
9   "left_fk": "renting_id",
10  "right_fk": "fine_id"
11 }
```

The response is a 201 CREATED status with a 15 ms response time and 351 B of data. The response body is a JSON object:

```
1 {
2   "join_table": "rentings_fines",
3   "left_id": 1,
4   "message": "Successfully created relation between rentings (id=1) and fines (id=2)",
5   "right_id": 2,
6   "status": "success"
7 }
```

FinesPayments:

The screenshot shows a REST client interface with a POST request to `{{base_url}} /api/stored-procedures/join_insert`. The request body is a JSON object with the following structure:

```
1 {
2   "left_table": "fines",
3   "left_lookup_col": "id",
4   "left_lookup_val": "2",
5   "right_table": "payments",
6   "right_lookup_col": "id",
7   "right_lookup_val": "2",
8   "join_table": "fines_payments",
9   "left_fk": "fine_id",
10  "right_fk": "payment_id"
11 }
```

The response is a 201 CREATED status with a 9 ms response time and 351 B of data. The response body is a JSON object:

```
1 {
2   "join_table": "fines_payments",
3   "left_id": 2,
4   "message": "Successfully created relation between fines (id=2) and payments (id=2)",
5   "right_id": 2,
6   "status": "success"
7 }
```

- c. Створити пакет, який вставляє 10 стрічок у довільну таблицю БД у форматі `<Noname+№>`, наприклад: `Noname5`, `Noname6`, `Noname7` і т.д. **(не реалістичне завдання враховуючи обмеження таблиць щодо обов'язкових значень, форматів значень, та зв'язків з іншими таблицями)**

- d. Написати **користувацьку функцію**, яка буде шукати Max, Min, Sum чи Avg для стовпця довільної таблиці у БД. Написати процедуру, яка буде у SELECT викликати цю функцію.

Payments:

POST {{base_url}} /api/stored-procedures/get-stat

Body

```
1 {
2   "table": "payments",
3   "column": "amount",
4   "stat_type": "AVG"
5 }
```

200 OK • 12 ms • 256 B

Body

```
1 {
2   "column": "amount",
3   "result": 10458.0,
4   "stat_type": "AVG",
5   "table": "payments"
6 }
```

Fines:

POST {{base_url}} /api/stored-procedures/get-stat

Body

```
1 {
2   "table": "fines",
3   "column": "amount",
4   "stat_type": "MAX"
5 }
```

200 OK • 7 ms • 252 B

Body

```
1 {
2   "column": "amount",
3   "result": 8500.0,
4   "stat_type": "MAX",
5   "table": "fines"
6 }
```

Users:

POST {{base_url}} /api/stored-procedures/get-stat

Body

```
1 {
2   "table": "users",
3   "column": "id",
4   "stat_type": "COUNT"
5 }
```

200 OK • 7 ms • 248 B

Body

```
1 {
2   "column": "id",
3   "result": 16.0,
4   "stat_type": "COUNT",
5   "table": "users"
6 }
```

е. Написати 1 процедуру із **курсором** для виконання однієї із наступних задач: Використовуючи курсор, забезпечити динамічне створення 2х таблиць з іменами що містять штамп часу, структура таблиць ідентична будь-якій структурі таблиці БД. Після чого випадковим чином пострічково скопіювати стрічки із батьківської таблиці або в одну, або в іншу додаткові таблиці. Повторний запуск процедури знову створює нові аналогічні таблиці, в яких випадковим чином знову будуть розкинуті дані з батьківської таблиці.

Users:

The screenshot shows a REST client interface with a POST request to `{{base_url}}/api/stored-procedures/split-table`. The request body is a JSON object: `{ "table": "users" }`. The response status is `201 CREATED` with a response time of 48 ms and a body size of 418 B. The response body is a JSON object: `{ "message": "Successfully split table users into 2 tables", "original_table": "users", "status": "success", "table1_name": "users_clone_1764516289_1", "table1_rows": 10, "table2_name": "users_clone_1764516289_2", "table2_rows": 6 }`.

Vehicles:

The screenshot shows a REST client interface with a POST request to `{{base_url}}/api/stored-procedures/split-table`. The request body is a JSON object: `{ "table": "vehicles" }`. The response status is `201 CREATED` with a response time of 35 ms and a body size of 430 B. The response body is a JSON object: `{ "message": "Successfully split table vehicles into 2 tables", "original_table": "vehicles", "status": "success", "table1_name": "vehicles_clone_1764516358_1", "table1_rows": 15, "table2_name": "vehicles_clone_1764516358_2", "table2_rows": 6 }`.

UserNotes:

The screenshot shows a REST client interface with a POST request to `{{base_url}}/api/stored-procedures/split-table`. The request body is a JSON object: `{ "table": "user_notes" }`. The response status is `201 CREATED` with a response time of 31 ms and a body size of 438 B. The response body is a JSON object: `{ "message": "Successfully split table user_notes into 2 tables", "original_table": "user_notes", "status": "success", "table1_name": "user_notes_clone_1764516383_1", "table1_rows": 12, "table2_name": "user_notes_clone_1764516383_2", "table2_rows": 9 }`.

4. Написати 3 довільні **тригери** для таблиць поточної БД, як приклад можна взяти наступні:
- Забезпечити кардинальність (min=2, max=6) стрічок для певної таблиці БД
 - Створити таблицю-журнал, в якій вести логи зі штампом часу при видаленні даних для певної таблиці
 - Для певного стовпця забезпечити формат вводу:
2 довільні букви, окрім M і R + '-' + 3 цифри + '-' + 2 цифри

[SOURCE](#)

```
18 def upgrade():
19     # '45000' - MySQL code for unhandled user-defined exception.
20     op.execute("""
21     CREATE TRIGGER trg_vehicle_plate_length_insert
22     BEFORE INSERT ON vehicles
23     FOR EACH ROW
24     BEGIN
25         IF CHAR_LENGTH(NEW.plate) < 2 OR CHAR_LENGTH(NEW.plate) > 10 THEN
26             SIGNAL SQLSTATE '45000'
27             SET MESSAGE_TEXT = 'Vehicle plate must be 2 to 10 characters long';
28         END IF;
29     END;
30     """)
31
32     op.execute("""
33     CREATE TRIGGER trg_vehicle_plate_length_update
34     BEFORE UPDATE ON vehicles
35     FOR EACH ROW
36     BEGIN
37         IF CHAR_LENGTH(NEW.plate) < 2 OR CHAR_LENGTH(NEW.plate) > 10 THEN
38             SIGNAL SQLSTATE '45000'
39             SET MESSAGE_TEXT = 'Vehicle plate must be 2 to 10 characters long';
40         END IF;
41     END;
42     """)
43
44     # ABE-123-456
45     # users.driver_license - format <3 letters (not M,R)> - <3 digits> - <3 digits>
46     op.execute("""
47     CREATE TRIGGER trg_driver_license_format_insert
48     BEFORE INSERT ON users
49     FOR EACH ROW
50     BEGIN
51         IF NEW.driver_license NOT REGEXP '^[A-LN-QS-Z]{3}-[0-9]{3}-[0-9]{3}$' THEN
52             SIGNAL SQLSTATE '45000'
53             SET MESSAGE_TEXT = 'Invalid driver_license format';
54         END IF;
55     END;
56     """)
57
58     op.execute("""
59     CREATE TRIGGER trg_driver_license_format_update
60     BEFORE UPDATE ON users
61     FOR EACH ROW
62     BEGIN
63         IF NEW.driver_license NOT REGEXP '^[A-LN-QS-Z]{3}-[0-9]{3}-[0-9]{3}$' THEN
64             SIGNAL SQLSTATE '45000'
65             SET MESSAGE_TEXT = 'Invalid driver_license format';
66         END IF;
67     END;
68     """)
69
70     # payments delete log
71     op.execute("""
72     CREATE TABLE payments_delete_log (
73         id INT AUTO_INCREMENT PRIMARY KEY,
74         deleted_payment_id INT NOT NULL,
75         deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
76     );
77     """)
78
79     op.execute("""
80     CREATE TRIGGER trg_payments_delete_log
81     AFTER DELETE ON payments
82     FOR EACH ROW
83     BEGIN
84         INSERT INTO payments_delete_log (deleted_payment_id)
85         VALUES (OLD.id);
86     END;
87     """)
88
```